République Tunisienne
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique Université de Sousse

Institut Supérieur des Sciences Appliquées et de Technologie de Sousse

DÉPARTEMENT INFORMATIQUE

RAPPORT DE MINI PROJET WEB

ING A2-G2-2

Quiz interactif en ligne

Élaboré par :

Étudiant 1 : Chaima Saadana
Étudiant 2 : Fedi Zarai

A.U. : 2026/2027

Plan

1. Introduction
2. Cahier des charges / Objectifs
3. Conception
4. Implémentation (code et explications)
5. Résultat final
6. Difficultés rencontrées et solutions
7. Conclusion
8. Annexes

1. Introduction

Ce rapport présente le mini-projet réalisé dans le cadre du TP de développement web (ING A2, département informatique de l'ISSATSo, année universitaire 2026/2027). Parmi les sujets proposés, nous avons choisi le sujet 7 : une page de quiz interactif.

L'objectif est de concevoir une interface web ludique qui pose une série de questions à l'utilisateur, chronomètre chaque réponse, corrige immédiatement par un retour visuel, calcule un score tenant compte du temps de réponse et conserve les meilleurs scores. Le projet utilise HTML5 pour la structure, CSS3 pour la mise en forme et les animations, et JavaScript pour la logique du quiz. Ce rapport décrit le cahier des charges, la conception, l'implémentation, le résultat obtenu et les difficultés rencontrées.

2. Cahier des charges / Objectifs

Le sujet demande de créer une interface web ludique permettant de répondre à un quiz et d'afficher les résultats en temps réel avec un retour visuel. Les contraintes techniques imposées sont les suivantes :

- HTML5, CSS3 et JavaScript ;
- design responsive avec Flexbox et Grid ;
- accessibilité et animations CSS ;
- validation W3C du code.

Les livrables attendus sont une page d'accueil et une interface de quiz, un retour visuel et un score final, des fichiers HTML, CSS et JS séparés, ainsi qu'une version responsive testée.

Fonctionnalités JavaScript demandées

La partie JavaScript comporte quatre fonctionnalités obligatoires et quatre bonus, que nous avons toutes réalisées :

Type	Fonctionnalité
Obligatoire	Chronomètre par question avec compte à rebours
Obligatoire	Calcul dynamique du score avec pénalités temporelles
Obligatoire	Feedback immédiat après chaque réponse (bonne ou mauvaise)
Obligatoire	Sauvegarde des meilleurs scores (localStorage)
Bonus	Rejouer avec réinitialisation complète
Bonus	Plusieurs types de questions (QCM et Vrai/Faux)
Bonus	Barre de progression visuelle
Bonus	Animation de transition entre les questions

3. Conception

Structure du dossier

```
/projet-quiz
├── index.html
├── style.css
└── script.js
```

Les trois fichiers sont séparés conformément au sujet : index.html contient la structure, style.css la mise en forme et script.js la logique du quiz.

Structure HTML

La page est organisée logiquement : un lien d'évitement pour l'accessibilité, un élément main qui contient trois sections (accueil, quiz, résultat) dont une seule est visible à la fois, puis un footer et l'appel au script. Le JavaScript affiche ou masque chaque section grâce à l'attribut hidden.

Règles du jeu et calcul du score

Le quiz comporte 10 questions (8 QCM et 2 Vrai/Faux) tirées dans un ordre aléatoire. Chaque question dispose de 15 secondes. Plus la réponse est rapide, plus elle rapporte de points :

Situation	Points
Bonne réponse immédiate	100
Pénalité de temps	– 5 points par seconde écoulée
Minimum pour une bonne réponse	25
Mauvaise réponse ou temps écoulé	0
Score maximum (10 questions)	1000

Données

Les questions sont stockées dans un tableau JavaScript (type, énoncé, options, indice de la bonne réponse et explication). Les cinq meilleurs scores (prénom, score, date) sont enregistrés dans le localStorage du navigateur, sous la clé « quiz-meilleurs-scores ».

Charte graphique

Nous avons choisi une ambiance ludique et lisible : un violet dominant, des cartes blanches aux coins arrondis, et des couleurs vives et sans ambiguïté pour le retour visuel. Les polices sont Baloo 2 pour les titres et Nunito pour le texte, toutes deux issues de Google Fonts.

Couleur	Code	Usage
Violet principal	#5B3DF5	Titres, boutons, barres de progression et de temps
Fond de page	#F1EFFC	Arrière-plan général
Vert	#12795A	Bonne réponse

Couleur	Code	Usage
Rouge	#B3203D	Mauvaise réponse, dernières secondes du chrono
Jaune	#FFC83D	Contour de focus au clavier

Outils utilisés

- Visual Studio Code avec l'extension Live Server ;
- navigateur Chrome et ses outils de développement (mode appareil mobile) ;
- validateurs W3C pour le HTML (validator.w3.org) et le CSS (jigsaw.w3.org/css-validator) ;
- Git et GitHub pour le dépôt du code ;
- Google Fonts pour les polices.

4. Implémentation (code et explications)

Exemple de code HTML

Voici la section du quiz. Elle contient le compteur, le chronomètre et le score, la barre de progression (avec les attributs ARIA), la barre de temps, la question, la liste des réponses remplie par JavaScript, le message de correction et le bouton de passage à la question suivante.

```
<section id="quiz" class="ecran" hidden>
    <div class="haut">
        <p id="compteur">Question 1 sur 10</p>
        <p id="chrono" aria-hidden="true">⏱️ 15 s</p>
        <p id="score-live">Score : 0 pts</p>
    </div>
    <div class="barre" role="progressbar" aria-label="Progression du quiz"
         aria-valuemin="0" aria-valuemax="10" aria-valuenow="0">
        <div id="remplissage"></div>
    </div>
    <div id="chrono-barre" class="chrono-barre" aria-hidden="true">
        <div id="chrono-remplissage"></div>
    </div>
    <h2 id="question" tabindex="-1">Chargement de la question</h2>
    <ul id="reponses" class="reponses"></ul>
    <p id="retour" class="retour" role="status"></p>
    <button id="suivante" class="bouton" type="button" hidden>
        Question suivante</button>
</section>
```

L'attribut role="status" permet aux lecteurs d'écran d'annoncer la correction, et tabindex="-1" permet de placer le focus sur la question à chaque changement. Le compteur de secondes est en aria-hidden pour éviter qu'un lecteur d'écran ne l'annonce chaque seconde.

Exemple de code CSS

Les réponses sont disposées avec CSS Grid sur deux colonnes. Le retour visuel combine la couleur, une icône (✓ ou ✗) et une animation, pour ne pas dépendre de la couleur seule. La barre

de temps devient rouge dans les cinq dernières secondes, et une animation fait glisser chaque nouvelle question. Sur mobile, une media query passe la grille sur une seule colonne.

```
.reponses {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 12px;
}

.reponse.juste::after { content: "✓"; font-size: 1.4rem; }
.reponse.faux::after  { content: "✗"; font-size: 1.4rem; }

#chrono-remplissage { height: 100%; width: 100%;
    background: var(--violet);
    transition: width .1s linear, background .3s; }
.chrono-barre.urgent #chrono-remplissage { background: var(--faux); }

.anim { animation: glisse .35s ease-out; }
@keyframes glisse {
    from { opacity: 0; transform: translateX(24px); }
    to   { opacity: 1; transform: none; }
}

@media (max-width: 600px) {
    .reponses { grid-template-columns: 1fr; }
}
```

Exemples de code JavaScript

Le chronomètre. La fonction lancerChrono() relance un décompte de 15 secondes : toutes les 100 millisecondes, elle calcule le temps restant à partir de l'horloge du navigateur, met à jour l'affichage et la barre, et déclenche une réponse « vide » lorsque le temps est écoulé.

```
function lancerChrono(courante) {
    arreterChrono();
    debut = performance.now();
    majChrono(DUREE);
    minuteur = setInterval(() => {
        const reste = Math.max(0, DUREE - (performance.now() - debut) / 1000);
        majChrono(reste);
        if (reste <= 0) repondre(null, courante);   // temps écoulé
    }, 100);
}
```

Le calcul du score. Dans la fonction repondre(), on arrête le chronomètre, on mesure le temps écoulé et on retire 5 points par seconde (avec un minimum de 25 points) pour une bonne réponse. Une réponse fausse ou un temps écoulé ne rapporte rien. Le paramètre bouton vaut null quand le temps est écoulé.

```
function repondre(bouton, courante) {
    if (repondu) return;
    repondu = true;
    arreterChrono();
```

```
    const ecoule = Math.floor((performance.now() - debut) / 1000);
    const tempsEcoule = bouton === null;
    const juste = !tempsEcoule && bouton.dataset.juste === "true";

    const gain = juste ? Math.max(POINTS_MIN, POINTS_MAX - PENALITE * ecoule) :
0;
    if (juste) { score += gain; bonnes++; }
    // ... coloration des boutons et message de correction
}
```

La sauvegarde des meilleurs scores. À la fin de la partie, le score est ajouté à la liste lue dans le localStorage, la liste est triée par ordre décroissant et seuls les cinq meilleurs sont conservés. Les lectures et écritures sont protégées par try/catch, car le stockage peut être indisponible.

```
function enregistrerScore() {
    const scores = lireScores();
    const record = score > 0 && (scores.length === 0 || score > scores[0].score);
    scores.push({
        nom: $("pseudo").value.trim() || "Anonyme",
        score,
        date: new Date().toLocaleDateString("fr-FR")
    });
    scores.sort((a, b) => b.score - a.score);
    ecrireScores(scores.slice(0, 5));
    afficherScores();
    return record;
}
```

L'ordre des questions et des réponses est mélangé à chaque partie avec l'algorithme de Fisher-Yates, sauf pour les questions Vrai/Faux dont les deux choix gardent leur ordre. Le bouton « Rejouer » relance simplement une partie : le chronomètre, le score et les compteurs sont remis à zéro.

5. Résultat final

La page d'accueil présente le quiz en une carte centrée : titre, description, informations clés, champ facultatif pour le prénom, bouton de démarrage et tableau des meilleurs scores.



Figure 1 : page d'accueil avec les meilleurs scores

Pendant le quiz, le compteur de questions, le temps restant, le score en points et deux barres (progression et temps) se mettent à jour. Une bonne réponse s'affiche en vert avec une coche et indique les points gagnés.

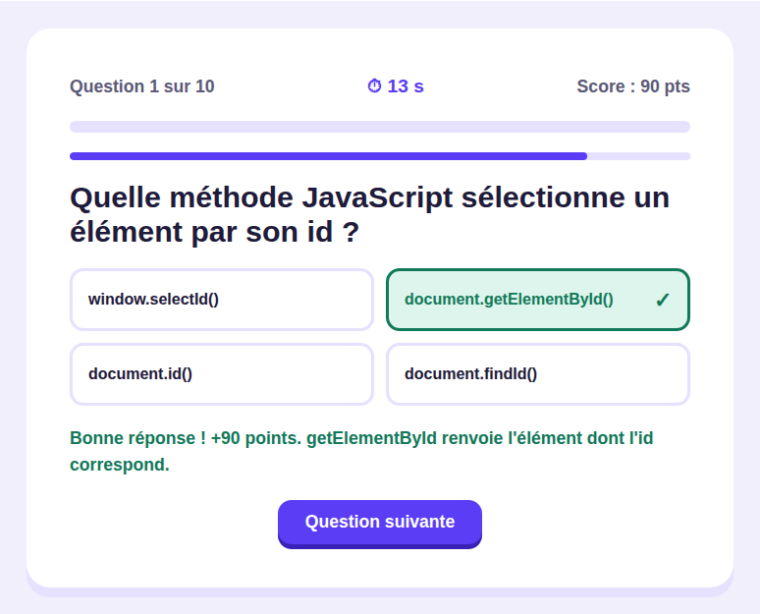


Figure 2 : bonne réponse (+90 points)

En cas d'erreur, la réponse choisie passe en rouge avec une croix, la bonne réponse est mise en évidence en vert, et le message indique la solution avec une courte explication. Les questions Vrai/Faux n'ont que deux boutons.



Figure 3 : mauvaise réponse sur une question Vrai/Faux

Dans les cinq dernières secondes, la barre de temps devient rouge. Si le temps est écoulé, la question est comptée comme ratée et la bonne réponse est affichée.



Figure 4 : dernières secondes, barre de temps rouge



Figure 5 : temps écoulé

À la fin, un cercle de progression en CSS affiche le score en points, accompagné du nombre de bonnes réponses, d'un message adapté au résultat (« Nouveau record ! » le cas échéant) et du classement des meilleurs scores.

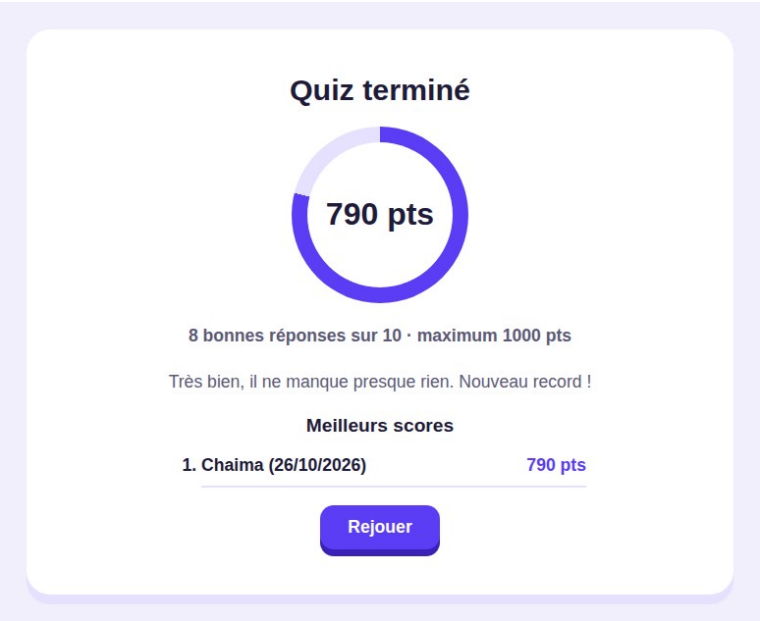


Figure 8 : score final et meilleurs scores

La mise en page s'adapte à la taille de l'écran : les réponses passent sur une seule colonne sur mobile, et la carte reste centrée et lisible sur tablette, avec la même charte graphique.



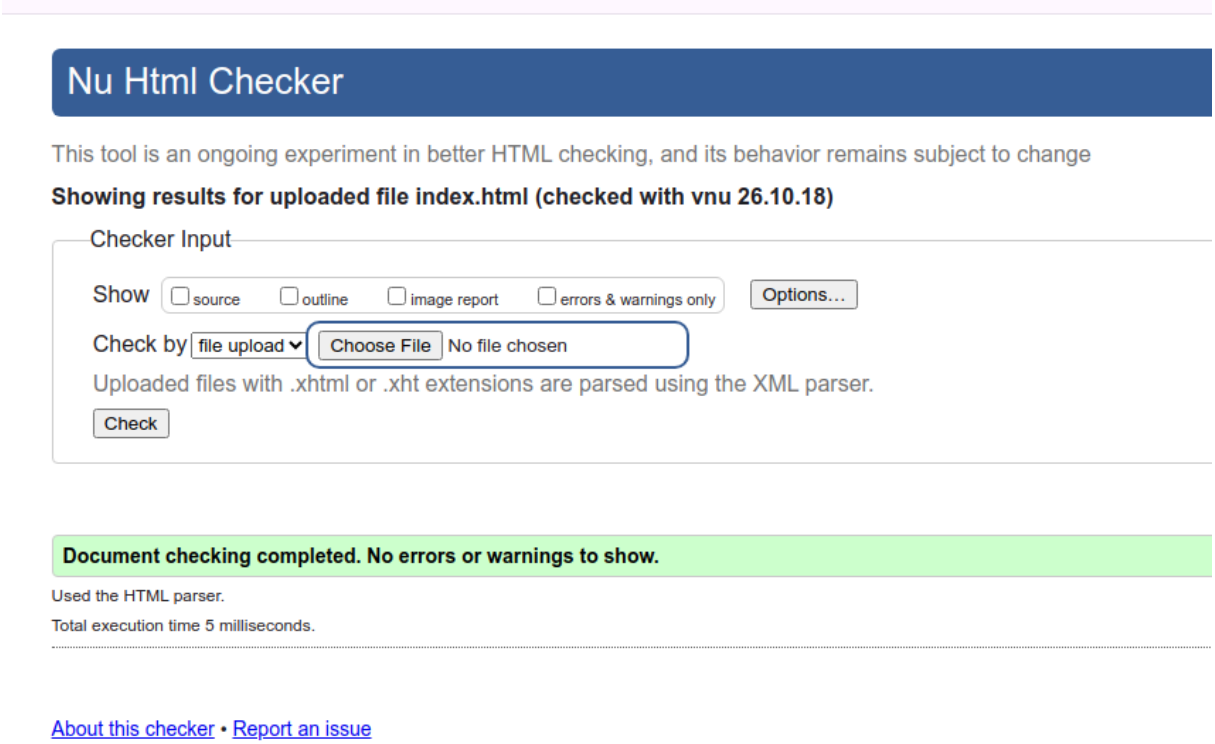
Figure 6 : version mobile (375 px)



Figure 7 : version tablette (768 px)

Validation W3C

Le fichier index.html a été vérifié avec le Nu Html Checker, le moteur de validation utilisé par validator.w3.org : aucune erreur et aucun avertissement. L'avertissement « Empty heading » signalé lors de notre première vérification a été corrigé (voir la section 6).



Résultat de la validation du fichier style.css avec le validateur CSS du W3C :



Résultats de la validation W3C CSS de style.css (CSS niveau 3 + SVG)

Félicitations ! Aucune erreur trouvée.

Ce document est valide conformément à la recommandation CSS niveau 3 + SVG !

Pour montrer à vos lecteurs votre souci d'interopérabilité lors de la création de cette page Web, vous pouvez a page Web:

```
<p>
    <a href="https://jigsaw.w3.org/css-validator/check/referer">
        <img style="border:0;width:88px;height:31px"
            src="https://jigsaw.w3.org/css-validator/images/vcss"
            alt="CSS Valide !" />
    </a>
</p>
```

```
<p>
    <a href="https://jigsaw.w3.org/css-validator/check/referer">
        <img style="border:0;width:88px;height:31px"
            src="https://jigsaw.w3.org/css-validator/images/vcss-blue"
            alt="CSS Valide !" />
    </a>
</p>
```

6. Difficultés rencontrées et solutions

- Avertissement W3C « Empty heading » : la balise <h2 id="question"> était vide dans le HTML car elle est remplie par JavaScript. Problème résolu en ajoutant un texte par défaut dans la balise, aussitôt remplacé par le script.
- Chronomètre : sans précaution, le décompte d'une question continuait de tourner après la réponse et pouvait provoquer des réponses en double. Résolu en arrêtant le minuteur avec clearInterval dans la fonction arreterChrono(), appelée à chaque réponse et à chaque nouvelle partie.
- Rendu des réponses sur mobile : deux colonnes étaient trop étroites sur petit écran. Corrigé avec une media query (max-width: 600px) qui passe la grille sur une colonne.
- Sauvegarde des scores : le localStorage peut être indisponible, et le prénom saisi par l'utilisateur ne doit pas pouvoir injecter de code. Résolu avec des blocs try/catch autour des lectures et écritures, et avec textContent plutôt que innerHTML pour afficher les noms.
- Réponses multiples : un second clic pouvait modifier le score. Résolu avec une variable (repondu) qui verrouille la question après la première réponse.

7. Conclusion

Ce projet nous a permis de consolider nos connaissances en HTML, CSS et JavaScript : structuration d'une page en plusieurs écrans, mise en page avec Flexbox et Grid, conception

responsive, animations CSS, manipulation du DOM, gestion du temps avec setInterval et persistance de données avec le localStorage. Nous avons aussi appris à tenir compte de l'accessibilité (navigation au clavier, focus visible, annonces pour lecteurs d'écran) et à valider le code avec les outils du W3C.

Il constitue une base solide pour aborder le développement web dynamique : le quiz pourrait par exemple être amélioré avec une base de questions chargée depuis un serveur ou un classement partagé entre plusieurs joueurs.

8. Annexes

- Code complet du projet : fichiers index.html, style.css et script.js (dossier projet-quiz).
- Dépôt GitHub : https://github.com/chaimasaadana/mini-projet-quiz.git
- Validateur HTML du W3C : https://validator.w3.org
- Validateur CSS du W3C : https://jigsaw.w3.org/css-validator
- Google Fonts (Baloo 2, Nunito) : https://fonts.google.com
- Documentation web MDN : https://developer.mozilla.org